Order Is a Superpower

Lecture Notes — Week 8: Searching & the Elementary Sorts

Auqib Hamid Lone

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

October 15, 2026

Concepts That Click

8.1 Why This Week Exists: What Order Buys a Search

Read alongside: Karumanchi, *Data Structures and Algorithms Made Easy*, Ch. 11, pp.547–589 (PDF) — the search operation and its variants; Wirth, *Algorithms and Data Structures*, Sec. 1.9.1–1.9.2, pp.34–36 — linear and binary search.

Four questions organize this chapter, stated here exactly as the deck states them: unordered or sorted — how much does order actually buy a search; what precondition does binary search silently assume, and what happens when it is violated; can we search for a value we never stored — the integer square root of n; and insertion, selection, bubble, counting — which sort does each input reward, and at what cost. Compressed to a plan: write the search contract, price linear versus binary, search a value that is not in any array at all, then sort — four algorithms, their traces, and their honest bills.

The handoff from Week 7 is exact. Week 7 finished the chain family: the doubly linked list (`p->prev`, the two-link invariant), circular rings that stop at `head` instead of `NULL`, the stack and queue rebuilt on nodes, and polynomials as ordered (coeff, exp) lists. It closed on one promise — "Searching — linear versus binary, and the elementary sorts that put data in the order binary search needs." The deficiency it left behind is the whole motivation: *every* lookup so far has been a walk. The calculator's symbol table — a list since Week 6 — answers a lookup by scanning from `head`, $O(n)$ worst case. New symbols today are the search bounds `lo`, `hi`, `mid`; the sort properties **stable** and **in-place**; and $T(n)$ for the four elementary sorts.

The week's job, in one line: show that *order* turns an $O(n)$ scan into an $O(\log n)$ probe — then learn the cheapest ways to create that order.

The running example: the calculator looks up an identifier

The tokenizer has just produced the identifier `x`, and the symbol table must answer: *is `x` declared, and where?* Held as a linked list (Week 6), each lookup scans from `head` until it either matches or hits `NULL` — $O(n)$ comparisons. For a program with 2,000 identifiers, that is up to 2,000 comparisons *per token*. If the same entries are kept *sorted by name*, however, a binary search answers in about $\log_2 2{,}000 \approx 11$ comparisons — roughly 180× fewer. Two questions follow immediately: how much does sorted order speed up search, and how do we get data sorted without paying more than we save?

Running application (Weeks 3–10): one calculator, rebuilt each week. Week 8 asks what its identifier table should do to look things up fast.

8.2 The Search Contract: Linear versus Binary

Read alongside: Karumanchi, Ch. 11, pp.547–589 (PDF) — the search operation, linear search, and its binary-search variants; Wirth, Sec. 1.9.1, p. 34 — linear search; Sec. 1.9.2, pp.34–36 — binary search.

8.2.1 The search operation and its precondition

Finding a name in an *unordered* pile means checking one item at a time. Finding an entry in a *sorted* directory lets you jump to the middle and discard half at a glance. The difference is not cleverness — it is a precondition.

Definition (The search operation (Karumanchi Ch. 11, pp.547–589 PDF; Wirth Sec. 1.9.1, p. 34)). `search(A, n, key)` *returns an index* i *with* `A[i] == key`*, or* -1 *if* `key` *does not occur.* ***Linear search*** *makes no assumption about the array.* ***Binary search*** requires *the array to be sorted in nondecreasing order — this precondition is part of the contract, not an optional extra. [2, 5]*

Example 8.2.1 (Why the precondition is the whole story)**.** *Work the deck's opening pair of arrays. In the unordered* `[7, 3, 9, 1, 5, 8]`*, searching for the value* 9 *can only be done the way linear search works — scan left to right (*`7`*, no;* `3`*, no;* `9`*, hit at index* 2*; three comparisons). In the sorted* `[1, 3, 5, 7, 8, 9]`*, the same target* 9 *can be found by halving: probe the middle (*`5` *at index* 2*), see that* $9 > 5$*, and keep only the right half* `[7, 8, 9]`*; probe its middle (*`8`*), keep* `[9]`*; found. The values are identical; only the* order *differs, and only the ordered one admits the shortcut.*

8.2.2 Linear search: scan until you find it

The deck's linear search, written in C (Karumanchi Ch. 11, pp.547–589 PDF):

```
int lsearch(int A[], int n, int key) {
  for (int i = 0; i < n; i++)
    if (A[i] == key) return i;   // hit: return the index
  return -1;                     // scanned the whole array: miss
}
```

Every element is tested at most once. The loop touches `A[0]`, then `A[1]`, and so on, in index order — no assumption about the values is ever used, which is exactly why it works on any input. Its cost depends only on *where* the key sits:

case	comparisons	example input
best	1	the first element matches
worst	n	absent, or last element
average (uniform random hit)	$(n+1)/2$	the key is equally likely anywhere

All three are $\Theta(n)$: linear search is a linear scan, and no ordering can change that. It is the honest baseline — and the price we pay for not owning sorted data.

Example 8.2.2 (Worked trace: linear search of `[7, 3, 9, 1, 5, 8]` for `key = 1`)**.** *The deck draws this trace.* `i = 0: A[0] = 7` $\neq 1$*.* `i = 1: A[1] = 3` $\neq 1$*.* `i = 2: A[2] = 9` $\neq 1$*.* `i = 3: A[3] = 1` $=$ *key — return* 3*. That is four comparisons. The walk is unavoidable without order: nothing in the array tells us that* 1 *cannot still be anywhere ahead, so each and every cell must be checked. See Figure 8.1.*

Concepts That Click

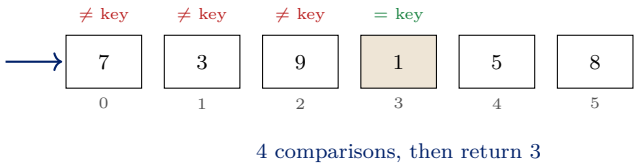


Figure 8.1: Linear search of `[7, 3, 9, 1, 5, 8]` for `key = 1`: three misses, then the hit at index 3 — four comparisons (cf. Karumanchi, Ch. 11, pp.547–589 PDF).

8.2.3 The sorted precondition — and what its absence costs

Binary search compares the target with the *middle* element, then keeps only the half that could contain it. That logic is sound *only if* everything left of the middle is smaller and everything right of it is larger — exactly the sorted invariant.

Definition (Binary search precondition (Karumanchi Ch. 11, pp.547–589 PDF)). *Binary search on $A[0..n-1]$ requires $A[0] \le A[1] \le \cdots \le A[n-1]$. Applied to unsorted data it does not error — it returns a* plausible wrong answer*, silently discarding the half that really held the key. The precondition is a* contract*, not a hint: binary search is only as correct as the order it is handed. [2]*

Example 8.2.3 (Worked failure trace: searching 1 in the unsorted `[7, 3, 9, 1, 5, 8]`)**.** *`lo = 0`, `hi = 5`, so `mid = 0 + (5 - 0)/2 = 2`, and `A[2] = 9`. Since $1 < 9$, the algorithm keeps the left half and sets `hi = 1`, discarding indices* 3..5 *— including the 1 at index 3. Second iteration: `lo = 0`, `hi = 1`, `mid = 0 + (1 - 0)/2 = 0`, and `A[0] = 7 > 1`, so `hi = -1`. Now `lo > hi`, the range is empty, and the search returns -1: a miss, even though the* 1 *is present. The precondition is a contract, not a hint.*

8.2.4 Binary search: halve the range

The deck's binary search, written in C (Wirth Sec. 1.9.2, pp.34–36; Karumanchi Ch. 11, pp.547–589 PDF):

```
int bsearch(int A[], int n, int key) {
  int lo = 0, hi = n - 1;                 // current [lo, hi], inclusive
  while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;         // avoids lo+hi overflow
    if (A[mid] == key) return mid;        // hit
    else if (A[mid] < key) lo = mid + 1; // key is in the right half
    else hi = mid - 1;                    // key is in the left half
  }
  return -1;                              // range empty: miss
}
```

Each iteration either stops or halves the live range $[\texttt{lo}, \texttt{hi}]$. Writing `lo + (hi - lo) / 2` rather than `(lo + hi) / 2` is not cosmetic: for large indices the sum can overflow `int` — Week 1's caution about C arithmetic in disguise.

Example 8.2.4 (Worked trace: the range shrinks)**.** *On a range of eight slots $[0, 8)$, the deck's diagram rows record the halving. Row 1: the full range* 0..7*, the blue rule marking `mid` at the slot whose right edge is $x = 4$, i.e. index* 3*. Row 2: the range* 0..3 *(the left half), `mid` at index* 1*. Row 3: the range* 2..3*, `mid` at index* 2*. Row 4: the range collapses to the single slot found at index* 3*. Each*

Concepts That Click

step discards roughly half the surviving range — that repeated halving is the whole mechanism. See Figure 8.2.

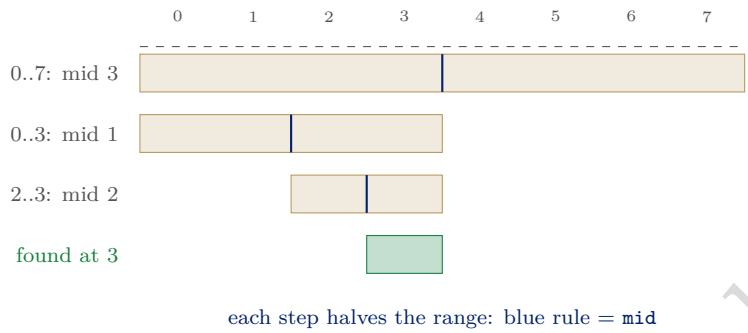


Figure 8.2: Binary search: each row halves the surviving range; the blue rule marks `mid` (cf. Wirth Sec. 1.9.2, pp.34–36).

8.2.5 Why binary search is $O(\log n)$

Let $T(n)$ be the worst-case number of comparisons for a range of size n. One comparison is spent, and then a range of size at most $\lceil n/2 \rceil$ remains, so the cost obeys

$$T(n) = T(\lceil n/2 \rceil) + 1, \qquad T(1) = 1.$$

Unrolling the recurrence: after k halvings the range has size $n/2^k$, which reaches 1 when $k = \log_2 n$. Hence $T(n) = \Theta(\log n)$ in the worst and average cases, and $O(1)$ in the best case (the target happens to be the middle element at once). The growth is unusually gentle:

n	10	100	1,000	1,000,000
comparisons $\approx \log_2 n$	4	7	10	20

Sedgewick classifies binary search as the canonical *logarithmic* order of growth ([4], order-of-growth table, p. 187; cost model, Sec. 1.4, p. 172). Doubling n adds just one comparison.

8.2.6 Binary-search variants: first and last occurrence

The plain search returns *some* matching index. To find the *first* occurrence — useful for counting duplicates — keep the matching slot as a candidate instead of returning it, and continue into the left half:

```
int lowerBound(int A[], int n, int key) {   // first i with A[i] >= key
  int lo = 0, hi = n;                        // half-open [lo, hi)
  while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (A[mid] < key) lo = mid + 1;          // discard [lo, mid]
    else hi = mid;                           // keep mid as a candidate
  }
  return lo;                                 // n if every element < key
}
```

Concepts That Click

The occurrence count of `key` in a sorted array is then `upperBound` − `lowerBound`, computed in $O(\log n)$ — no scan (the upper bound is the same routine with $>$ in place of $\geq$; Karumanchi Ch. 11, pp.547–589 PDF). The one line that matters is `hi = mid` (rather than `hi = mid - 1`): keeping `mid` as a candidate is precisely what pinpoints the *first* match instead of discarding it.

8.2.7 A prediction to check: the unsorted array

Example 8.2.5 (Check your prediction — the unsorted array, with the deck's answer)**.** *The array* `[7, 3, 9, 1, 5, 8]` *is unsorted. You run binary search for the key* 3*. What does it return? (A) index* 1*; (B) a wrong index or* −1*; (C) a runtime error. The deck's stated answer is* ***B****:* `mid = 2` *(value* 9*);* $3 < 9$*, so it discards indices* 3..5 *and continues left; the range collapses to* $\{0\}$*, the value* 7 *there is* > 3*, and the search returns* −1 *— silently losing the* 3 *that sits at index* 1*. Option C is wrong because the code compiles and runs — nothing checks the precondition.*

8.2.8 Search cost, side by side

algorithm	precondition	best	worst	space
linear search	none	$O(1)$	$\Theta(n)$	$O(1)$
binary search (iterative)	sorted	$O(1)$	$\Theta(\log n)$	$O(1)$

Binary search's $\Theta(\log n)$ worst case looks strictly better — and it is, *once you own sorted data*. The catch is the precondition: it costs something to establish, and it must be maintained after every insertion. The rest of this chapter is about that cost.

8.3 Binary Search Beyond Arrays: Two Applications

Read alongside: Karumanchi, Ch. 11, pp.547–589 (PDF) — search on a monotone predicate; Ch. 15, pp.640–684 (PDF) — string search.

Binary search is not only for arrays of stored data. Whenever a yes/no question about a value has a single true→false flip point (a *monotone predicate*), binary search finds that point in $O(\log(\text{range}))$ probes — even when no element of an array is involved. Two applications make the pattern concrete.

8.3.1 Application 1: the integer square root of n

Suppose we want $\lfloor\sqrt{n}\rfloor$ — the largest integer k with $k^2 \leq n$ — using only integer arithmetic, no floating point. The answer k is not stored in any array. The trick is to search the *answer space* $0..n$ instead of an array: define the predicate $P(k) = (k^2 \leq n)$. As k grows, P is *true* up to a point and *false* after it — a single true→false boundary. A monotone predicate is exactly what binary search needs: find the largest k with $P(k)$ true, and that k is $\lfloor\sqrt{n}\rfloor$.

The deck's code (standard treatment; Karumanchi Ch. 11, pp.547–589 PDF):

```
int isqrt(int n) {                      // floor(sqrt(n)), n >= 0
  int lo = 0, hi = n, ans = 0;
  while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if ((long long) mid * mid <= n) {     // P(mid) true
      ans = mid; lo = mid + 1;            // mid is feasible; push right
    } else hi = mid - 1;                  // P(mid) false; push left
  }
```

Concepts That Click

```
  return ans;                              // largest feasible k
}
```

The `(long long)` cast widens *before* the multiply: `mid` can reach n, and n^2 overflows `int` — and even a 32-bit `long` on Windows (LLP64). Week 1's arithmetic caution again. The cost is $O(\log n)$ probes and $O(1)$ space: the range $0..n$ halves each step, no array, no floating point.

Example 8.3.1 (Worked example: $\lfloor\sqrt{27}\rfloor$)**.** *The boundary sits where k^2 first exceeds 27. Tabulate $P(k) = (k^2 \le 27)$:*

k	*0*	*1*	*2*	*3*	*4*	*5*	*6*	*7*
k^2	*0*	*1*	*4*	*9*	*16*	*25*	*36*	*49*
$P(k)$	*T*	*T*	*T*	*T*	*T*	*T*	*F*	*F*

Since $25 \le 27 < 36$, the boundary sits at $k = 5$: $P(5)$ is the last true value, $P(6)$ the first false. Binary search returns the largest true k, so $\lfloor\sqrt{27}\rfloor = 5$, found in $O(\log 27)$ probes. See Figure 8.3.

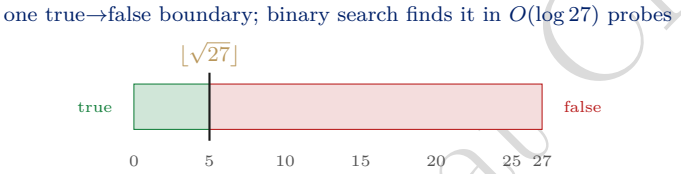


Figure 8.3: The square-root search space: $P(k) = (k^2 \le 27)$ is true for $k \le 5$ and false after, and binary search finds the single boundary (cf. Karumanchi, Ch. 11, pp.547–589 PDF).

8.3.2 Application 2: the longest common prefix

The calculator's autocomplete must find the longest prefix shared by all identifiers starting with a typed stem. For "flower", "flow", "flight" the shared prefix is "fl". Two facts make this a binary-search problem:

1. If the strings are *sorted*, the longest common prefix of the whole set equals the common prefix of the first and the last string — so only two strings need comparing.
2. The length L of the common prefix of two strings is itself found by binary search: the predicate "the first L characters are equal" is true for small L and false beyond the answer — again a single true→false boundary.

So: sort the set once ($O(n \log n \cdot m)$ for strings of length up to m), then answer each prefix query in $O(m)$ using the first and last strings — no scan over all n strings. The deck's code (standard treatment; Karumanchi Ch. 15, pp.640–684 PDF, string search):

```
int lcp2(const char *s, const char *t) {   // common prefix of s, t
  int lo = 0, hi = min(strlen(s), strlen(t)), ans = 0;
  while (lo <= hi) {
    int L = lo + (hi - lo) / 2;
    if (prefixEq(s, t, L)) { ans = L; lo = L + 1; }  // P(L) true
    else hi = L - 1;                                  // P(L) false
  }
  return ans;
}
// For a SORTED set: LCP of all = lcp2(first, last).
```

Concepts That Click

`prefixEq(s, t, L)` costs $O(L)$; trying $O(\log m)$ candidate lengths makes the query $O(m \log m)$ — or $O(m)$ if each check is made incremental. The sorted-set fact shortens the set-side work: the LCP of *all* strings equals the LCP of the first and the last, so there is no $O(n)$ scan.

Example 8.3.2 (Worked example: LCP of `{"flight", "flow", "flower"}`)**.** *The set is already in sorted order, so its overall common prefix equals the common prefix of the first and last strings,* `"flight"` *and* `"flower"`*. Compare character by character: position* 0 *is "f" in both; position* 1 *is "l" in both; position* 2 *is "i" in* `"flight"` *but "o" in* `"flower"` *— the first disagreement. The shared prefix is therefore "fl", of length* 2*. Checking all three strings against "fl" is unnecessary once they are sorted: only* `flight` *and* `flower` *need comparing, and their common prefix* is *the set's. See Figure 8.4.*

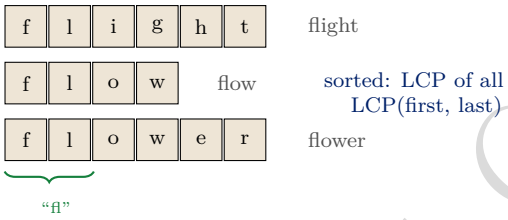


Figure 8.4: The longest common prefix of a sorted set equals the LCP of its first and last strings: "fl" here (cf. Karumanchi, Ch. 15, pp.640–684 PDF).

Example 8.3.3 (Check your prediction — prefix length, with the deck's answer)**.** *For the sorted set* `["flight", "flow", "flower"]`*, the predicate "all share a prefix of length* L*" is tested. For which* L *is it first* false*? (A)* $L = 1$*; (B)* $L = 2$*; (C)* $L = 3$*. The deck's answer is* ***C****. Length* 1 *is "f", shared by all three: true. Length* 2 *is "fl", still shared by all three: true, so the boundary lies* after *this value. Length* 3 *would require "fli" and "flo" to agree, but "flight" has an "i" where "flow" has an "o" — the first false at* $L = 3$*. The answer (the longest common length) is* 2*, the largest* L *that is still true; binary search returns* L_{true}*, and the prefix is its first* L *characters. Correct: C.*

8.4 Creating Order: The Four Elementary Sorts

Read alongside: Karumanchi, Ch. 10, pp.504–546 (PDF) — insertion, selection, bubble, and counting sort; Wirth, Sec. 2.2.1–2.2.3, pp.46–50 — straight insertion, selection, and exchange sort; Sedgewick, Sec. 2.1, p. 244 — elementary sorts.

8.4.1 Why sort at all?

Binary search is the reward; sorting is the price we pay for it. If the identifier table is sorted once and read many times, that price is amortized away. But sorting is not only for search: it puts "equal" things next to each other, which is how we deduplicate, find duplicates, group records, and compute medians. The calculator's symbol table, its autocomplete set, and its report of repeated declarations all begin with a sort.

Four algorithms are introduced here — not as a list to memorize but as four answers to the question "what is the cheapest way, given *this* input?":

Concepts That Click

sort	idea	signature
insertion	keep $A[0..i-1]$ sorted; insert $A[i]$ into place	adaptive: near-sorted input is fast
selection	find the minimum of the unsorted part; swap it forward	exactly $n-1$ swaps, always $\Theta(n^2)$ comparisons
bubble	swap adjacent inversions; the largest "bubbles" to the end	can stop early when a pass makes no swap

All three sorts above sort *by comparing pairs* of elements, so they work on any comparable type. Counting sort, introduced later in this section, breaks that property and gets linear time — but only for keys drawn from a small integer range. Two pieces of vocabulary decide many real choices:

Definition (Stable sort). *A sort is **stable** if equal keys keep their original relative order.*

Definition (In-place sort). *A sort is **in-place** if it uses $O(1)$ auxiliary space beyond the input.*

Reference treatment: Knuth, *TAOCP* Vol. 3 [3]; Horowitz & Sahni Ch. 7.

8.4.2 Insertion sort: grow a sorted prefix

The deck's insertion sort, written in C (Wirth Sec. 2.2.1, p. 46; Karumanchi Ch. 10, pp.504–546 PDF):

```
void insertionSort(int A[], int n) {
  for (int i = 1; i < n; i++) {
    int key = A[i], j = i - 1;          // key is the element to place
    while (j >= 0 && A[j] > key) {      // shift larger elements right
      A[j + 1] = A[j];
      j--;
    }
    A[j + 1] = key;                     // drop key into its slot
  }
}
```

The invariant is: before iteration i, the subarray $A[0..i-1]$ is sorted. The outer loop extends that sorted prefix by one element; the inner loop shifts the larger elements rightward until `key` finds its place. Each new key is slid left past the elements greater than it. When the prefix is already sorted, the inner loop runs *once* — which is the whole point of an adaptive sort.

Example 8.4.1 (Worked trace: insertion sort of `[5, 2, 4, 6, 1, 3]`). *Start with the prefix `[5]`. Then insert each remaining element into the sorted prefix:*

step	*key*	*shifted*	*array after*
1	2	5 *(one shift)*	`[2, 5, 4, 6, 1, 3]`
2	4	5 *(one shift)*	`[2, 4, 5, 6, 1, 3]`
3	6	*none* ($6 > 5$)	`[2, 4, 5, 6, 1, 3]`
4	1	$6, 5, 4, 2$ *(four)*	`[1, 2, 4, 5, 6, 3]`
5	3	$6, 5, 4$ *(three)*	`[1, 2, 3, 4, 5, 6]`

The gold prefix in Figure 8.5 grows by one each step. Step 3 is the adaptive case in miniature: the prefix was already in order, so the inner loop stopped after a single comparison and no shift.

Concepts That Click

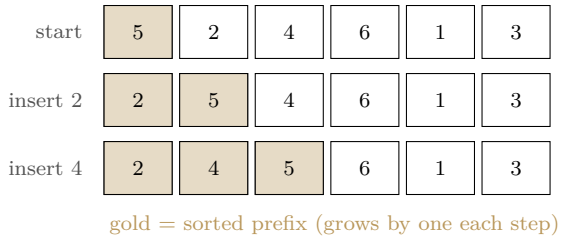


Figure 8.5: Insertion sort of `[5, 2, 4, 6, 1, 3]`: the gold sorted prefix grows by one each step (cf. Wirth Sec. 2.2.1, p. 46).

8.4.3 Insertion sort: the bill

Count the shifts (the inner-loop body). The outer loop runs $i = 1 \ldots n-1$, so there are $n-1$ insertions:

case	**comparisons / shifts**	**example input**
best	$n-1$ (each key already in place)	already sorted: $O(n)$
worst	$\sum_{i=1}^{n-1} i = \dfrac{n(n-1)}{2}$	reverse sorted: $\Theta(n^2)$
average	$\Theta(n^2)$	random order

Insertion sort is adaptive: its cost falls to $O(n)$ on nearly-sorted input — it is the only one of the three comparison sorts with a linear best case. It is also stable (equal keys stay in order, because we shift only on `A[j] > key`, not $\geq$) and in-place ($O(1)$ extra space). Those last two properties are why insertion sort is the base case inside efficient sorts in Week 9.

8.4.4 Selection sort: repeatedly pick the minimum

The deck's selection sort, written in C (Wirth Sec. 2.2.2, p. 48; Karumanchi Ch. 10, pp.504–546 PDF):

```
void selectionSort(int A[], int n) {
  for (int i = 0; i < n - 1; i++) {
    int m = i;                          // index of the minimum so far
    for (int j = i + 1; j < n; j++)
      if (A[j] < A[m]) m = j;           // find the smallest in A[i..n-1]
    swap(&A[i], &A[m]);                 // put it at the front of the rest
  }
}
```

The invariant is: after iteration i, the subarray $A[0..i]$ holds the $i+1$ smallest elements, in final position. The scan finds the next minimum, and one swap moves it into place. Note that the scan always covers the whole unsorted remainder — regardless of whether the input is already sorted.

Example 8.4.2 (Worked trace: selection sort of `[5, 2, 4, 6, 1, 3]`). *Row 1 of Figure 8.6: the scan over the whole array finds* 1 *at index* 4 *(the current minimum, gold); swap it with index* 0 *(value* 5*), giving* `[1, 2, 4, 6, 5, 3]`*. The first slot is now finalised (green). Row 2: the scan of* `A[1..5]` *finds* 2*; it is already at index* 1*, so the swap is a no-op. Row 3: the scan of* `A[2..5]` *finds* 3 *at index* 5*; swap it with index* 2 *(value* 4*), giving* `[1, 2, 3, 6, 5, 4]` *and finalising indices* 0..2*. The green region is the growing block of finalised smallest elements. See Figure 8.6.*

Concepts That Click

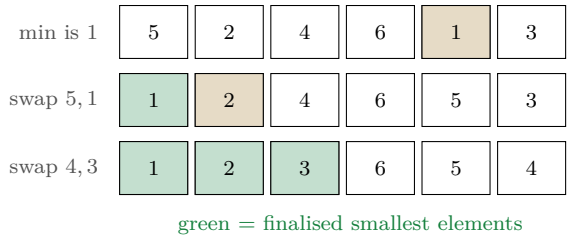


Figure 8.6: Selection sort of `[5, 2, 4, 6, 1, 3]`: green marks finalised smallest elements, gold the current minimum (cf. Wirth Sec. 2.2.2, p. 48).

8.4.5 Selection sort: the bill (and a warning)

The inner-loop comparisons do not depend on the data at all: $\sum_{i=0}^{n-2}(n-1-i) = \dfrac{n(n-1)}{2}$ — every input.

case	comparisons	swaps
best	$\Theta(n^2)$	0 (already sorted)
worst	$\Theta(n^2)$	$n-1$

Selection sort is not adaptive: a sorted input is scanned just as hard as a reversed one, always $\Theta(n^2)$ comparisons. Its one virtue is the minimum number of *writes* ($n-1$ swaps), useful when moving data is far costlier than comparing it. It is not stable: the long-range swap can leap an equal key over its twin (trace `[2, 2, 1]` to see the two 2s change order). It is, however, in-place: $O(1)$ auxiliary space.

8.4.6 Bubble sort: swap neighbours, bubble the max up

The deck's bubble sort with an early exit, written in C (Wirth Sec. 2.2.3, p. 50; Karumanchi Ch. 10, pp.504–546 PDF):

```
void bubbleSort(int A[], int n) {
  for (int i = 0; i < n - 1; i++) {
    int swapped = 0;
    for (int j = 0; j < n - 1 - i; j++)
      if (A[j] > A[j + 1]) { swap(&A[j], &A[j + 1]); swapped = 1; }
    if (!swapped) break;     // a clean pass: already sorted
  }
}
```

The invariant is: after pass i, the largest $i+1$ elements are final at the right end, so each pass can stop i slots earlier. The `swapped` flag is the practical touch: a pass with no swap proves the array is sorted, so we stop.

Example 8.4.3 (Worked trace: one bubble pass over `[5, 2, 4, 6, 1, 3]`). *Compare neighbours `(A[j], A[j+1])` and swap when out of order. `(5,2)`: $5 > 2$, swap ⇒ `[2, 5, 4, 6, 1, 3]`. `(5,4)`: swap ⇒ `[2, 4, 5, 6, 1, 3]`. `(5,6)`: in order, no swap. `(6,1)`: swap ⇒ `[2, 4, 5, 1, 6, 3]`. `(6,3)`: swap ⇒ `[2, 4, 5, 1, 3, 6]`. After one full pass the maximum* 6 *has "bubbled" to the end and is final. The next pass stops one slot sooner. See Figure 8.7.*

Concepts That Click

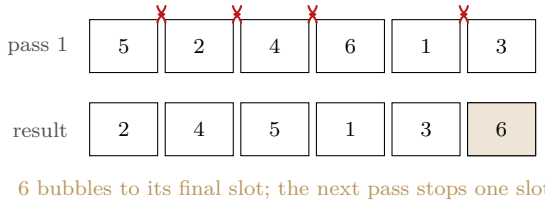


Figure 8.7: One bubble pass over `[5, 2, 4, 6, 1, 3]`: adjacent swaps push the maximum 6 to its final slot (cf. Wirth Sec. 2.2.3, p. 50).

8.4.7 Bubble sort: the bill

case	comparisons	note
best	$n-1$	sorted input: pass 1 swaps nothing, early exit
worst	$\frac{n(n-1)}{2}$	reverse sorted: $\Theta(n^2)$
average	$\Theta(n^2)$	random order

Bubble sort is adaptive only *with* the early-exit flag; the naive version is $\Theta(n^2)$ on every input, including sorted data. It is stable (equal adjacent keys are never swapped, because we swap only on strict $>$ — their relative order is preserved) and in-place. Its comparison count equals insertion sort's on average, but bubble sort does far more *swaps*; insertion sort is the better practical choice among the three.

All three comparison sorts hit the same $\Theta(n^2)$ wall. Week 9 asks whether any comparison sort can do better — and shows counting sort, next, sidestepping comparisons entirely.

8.4.8 Counting sort: sort without comparing

All three sorts above compare keys pairwise. But if the keys are small non-negative integers, the key *is already an array index* — so it can be counted, not compared. Counting sort tallies how many times each value occurs, converts those counts to positions with a running total, and scatters each element to its slot. It never asks "is $A[i] < A[j]$?" The precondition is that keys lie in a known small range $0..k-1$. The trade is linear time for $\Theta(n+k)$ extra space — and it is only correct for keys that fit that range.

In the calculator, counting the frequency of each byte or token class, and bucketing identifiers by length, are both one-pass counting sorts — no comparisons at all. The deck's code, with a stable scatter (Karumanchi Ch. 10, pp.504–546 PDF; Horowitz & Sahni Ch. 7, standard treatment):

```
void countingSort(int A[], int n, int k) {   // keys in 0..k-1
  int *count = calloc(k, sizeof(int));       // zero-initialised tally
  for (int i = 0; i < n; i++) count[A[i]]++;
  for (int v = 1; v < k; v++) count[v] += count[v - 1];  // prefix sums
  int *out = malloc(n * sizeof(int));
  for (int i = n - 1; i >= 0; i--)           // reverse pass => stable
    out[--count[A[i]]] = A[i];
  for (int i = 0; i < n; i++) A[i] = out[i];
  free(out); free(count);                    // Week 1: every malloc freed
}
```

[2, 1]

Concepts That Click

`calloc(k, ...)` zeroes the tally, so no counting slot is ever read before it is written. The reverse pass over A makes equal keys land in their original relative order — that is what makes this sort stable.

Example 8.4.4 (Worked trace: counting sort of `A = [4, 2, 2, 5, 3, 3, 1]`). *Keys lie in* 0..5, *so* $k = 6$. ***Tally:*** *value* 0 *occurs* 0 *times, value* 1 *once, value* 2 *twice, value* 3 *twice, value* 4 *once, value* 5 *once — the row* `[0, 1, 2, 2, 1, 1]`. ***Prefix sums:*** `count[0] = 0; count[1] = 0+1 = 1; count[2] = 1+2 = 3; count[3] = 3+2 = 5; count[4] = 5+1 = 6; count[5] = 6+1 = 7` — *the row* `[0, 1, 3, 5, 6, 7]`. *Each prefix entry is the position one past the last slot of that value: the* 1*st slot belongs to value* 0, *the slots* 1–2 *to value* 2, *and so on.* ***Scatter (reverse):*** *walking* A *from the end places each value in the slot given by* `--count[A[i]]`, *which fills each value's block right-to-left and thus preserves the order of equal keys. The result is sorted. See Figure 8.8.*

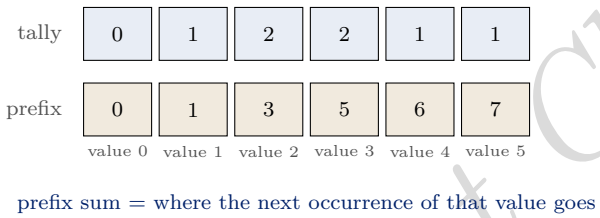


Figure 8.8: Counting sort of $A = [4, 2, 2, 5, 3, 3, 1]$: the tally row counts each value, the prefix row gives the scatter positions (cf. Karumanchi, Ch. 10, pp.504–546 PDF).

8.4.9 Counting sort: the bill

phase	work	cost
tally	one pass over A	$\Theta(n)$
prefix sums	one pass over $0..k-1$	$\Theta(k)$
scatter	one reverse pass over A	$\Theta(n)$
total		$\Theta(n+k)$ time, $\Theta(n+k)$ space

Counting sort is linear time when $k = O(n)$. But watch the range: if keys span a huge range (say $0..10^9$), the tally array is enormous and counting sort is worse than useless — this is why it is restricted to small integer keys. It is stable (with the reverse scatter) but not in-place ($\Theta(n+k)$ auxiliary). Crucially, it sorts *without comparing keys* — so the $\Theta(n \log n)$ comparison lower bound of Week 9 does not apply to it. [2]

8.4.10 The four sorts, compared

sort	best	worst	space	stable	in-place
insertion	$O(n)$	$\Theta(n^2)$	$O(1)$	yes	yes
selection	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	no	yes
bubble	$O(n)$	$\Theta(n^2)$	$O(1)$	yes	yes
counting	$\Theta(n+k)$	$\Theta(n+k)$	$O(n+k)$	yes	no

Read the table as a workload guide, not a ranking: **nearly sorted** input → insertion sort, whose $O(n)$ best case is the whole point; **small integer keys** in a known range → counting sort, linear but at the cost of $O(n+k)$ space; and when **minimal writes** matters more than comparisons

Concepts That Click

→ selection sort, with its exactly $n-1$ swaps. No single winner — every row is right for *some* input. The skill is stating the workload, then choosing.

Example 8.4.5 (Check your prediction — which sort? with the deck's answer)**.** *Two inputs. Which elementary sort fits each best? (i) an array that is already almost sorted; (ii)* 10^6 *ages in* 0..120*. The deck's answer: for (i),* ***insertion sort*** *— nearly every key is already in place, so the inner loop rarely shifts and the run is* $O(n)$*. For (ii),* ***counting sort*** *— the keys are integers in a tiny range, so* $k = 121 \ll n$ *and the sort is* $\Theta(n+k) \approx \Theta(n)$*. Selection or bubble would be* $\Theta(n^2)$ *on both — the wrong tool for either input. Matching the algorithm to the input's structure is the real skill this week tests, not memorizing the update loops.*

8.5 Stability, Classic Mistakes, and the Close

Read alongside: Karumanchi, Ch. 10, pp.504–546 (PDF) — sort properties and the comparison of elementary sorts (this review is the deck's own, not a new citation).

8.5.1 Stability and in-place, made concrete

Stable means equal keys keep their original relative order. This matters when records are sorted by a *secondary* key after already being sorted by a primary one — a stable sort preserves the earlier ordering within ties.

Example 8.5.1 (Worked example — stability)**.** *Sort* `[(A,2), (B,1), (C,2)]` *by the number. A stable sort yields* `[(B,1), (A,2), (C,2)]` *—* `A` *stays before* `C`*, since both carry key* 2 *and* `A` *came first. An unstable sort may return* `[(B,1), (C,2), (A,2)]` *— same keys, different record order.*

In-place means $O(1)$ auxiliary space beyond the input. Insertion, selection, and bubble sort are in-place; counting sort needs a $\Theta(n+k)$ output buffer and a $\Theta(k)$ tally, so it is not. Summing the properties: stable sorts here are insertion, bubble, and counting; not stable is selection, whose long-range swap can leap an equal key.

8.5.2 Three classic mistakes

Binary searching unsorted data — no crash, just a plausible wrong answer. The precondition is a contract: sort first, or use linear search. Forgetting the overflow in `mid` — `(lo + hi) / 2` overflows `int` for large indices; write `lo + (hi - lo) / 2`. The same trap appears as `mid * mid` in `isqrt` and the `(long long)` casts in C. Claiming a sort is $\Theta(n \log n)$ "because it sorts" — the elementary sorts are all $\Theta(n^2)$ worst case. State the *case* and the *bound*, and check whether the algorithm even compares keys (counting sort does not).

How to revise: for each sort, write one sentence naming the invariant, the best case, the worst case, and whether it is stable — then trace one pass by hand. The test is traces and cases, not definitions.

8.5.3 Week 8 summary

The search contract: linear search needs no order, $\Theta(n)$ worst case; binary search *requires* sorted input and is $\Theta(\log n)$ worst case — but is silently wrong on unsorted data. **Binary search on a monotone predicate** answers questions about values that are not stored: $\lfloor \sqrt{n} \rfloor$ (search the answer space $0..n$), and the longest common prefix (search the prefix length; sort once, then compare first and last). **Elementary sorts**: insertion (adaptive, stable, $O(n)$ best); selection (always $\Theta(n^2)$,

Concepts That Click

$n-1$ swaps, unstable); bubble (stable, adaptive with an early exit); counting ($\Theta(n+k)$, stable, $O(n+k)$ space, small integer keys). **Dictionary at the end**: stable, in-place, best/worst case — the three properties that decide which sort fits an input.

Next week: why can no *comparison* sort beat $\Theta(n \log n)$, and which algorithms escape that bound? Efficient sorting — quick, merge, shell, radix, heap.

Example 8.5.2 (Before-you-go predictions, with the deck's answers)**.** *Answer from the algorithms, then check in the lab. (1) Binary search on a sorted array of* 1,000 *elements: about* 10 *comparisons in the worst case (*$\log_2 1000$*), against up to* 1,000 *for linear search — order is worth roughly a hundredfold here. (2) Sorting* `[3, 1, 3, 2]`*: insertion, bubble, and counting are stable; selection may reorder equal keys. (3) Yes, the "first occurrence" variant of binary search differs from ordinary binary search: keeping* `mid` *as a candidate when* `A[mid] == key` *lets the search continue left to find the* first *match.*

Exercises

Answer from the algorithms, then check against the Solutions block: search and sort traces run on paper first.

1. **The failed precondition.** Trace binary search (the `bsearch` code above) on the unsorted array `[7, 3, 9, 1, 5, 8]` for the key 1: give `lo`, `hi`, and `mid` at each iteration, and the final return value. Then explain why the algorithm never notices the violation.

2. **Integer square root.** Use the `isqrt` algorithm to compute $\lfloor\sqrt{50}\rfloor$ without any floating point: list the predicate $P(k) = (k^2 \le 50)$ at the probed values, and give the returned value. (Genuinely new instance.)

3. **Insertion sort.** Trace insertion sort on `[4, 2, 5, 1, 3]`: after each outer iteration i, write the array, and give the total number of shifts. (Genuinely new instance.)

4. **Counting sort.** Sort $A = [2, 0, 3, 2, 1, 3, 0]$ with keys in 0..3: give the tally row, the prefix-sum row, the value of k, and the final sorted array. Which property does the reverse scatter protect? (Genuinely new instance.)

5. **Stability.** For `[3, 1, 3, 2]` (label the two 3s as 3_a then 3_b), say which of insertion, selection, bubble, and counting sort are stable, and give the tie order a stable sort produces.

Solutions

1. **Iteration 1:** `lo = 0`, `hi = 5`, `mid = 2`, `A[2] = 9 > 1`, so `hi = 1`. **Iteration 2:** `lo = 0`, `hi = 1`, `mid = 0`, `A[0] = 7 > 1`, so `hi = -1`. Now `lo > hi`; return -1 (a miss, though 1 is present at index 3). The algorithm never notices because the code has no check of whether the input is sorted — it simply trusts the precondition, and a wrong "greater-than" comparison discards the half that held the key.

2. $\lfloor\sqrt{50}\rfloor$**:** $7^2 = 49 \le 50$, $8^2 = 64 > 50$, so the boundary sits at $k = 7$ and the answer is 7. (Binary search probes the answer space 0..50 and returns the largest k with $P(k)$ true.)

3. After $i = 1$ (key 2): `[2, 4, 5, 1, 3]`. After $i = 2$ (key 5): `[2, 4, 5, 1, 3]`, no shift ($5 > 4$). After $i = 3$ (key 1): shift $5, 4, 2 \Rightarrow$ `[1, 2, 4, 5, 3]`. After $i = 4$ (key 3): shift $5, 4 \Rightarrow$ `[1, 2, 3, 4, 5]`. Total shifts: $1 + 0 + 3 + 2 = 6$.

Concepts That Click

4. Keys in 0..3, so $k = 4$. **Tally** (values 0..3): `[2, 1, 2, 2]`. **Prefix sums:** `count[0]=2`; `count[1]=2+1=3`; `count[2]=3+2=5`; `count[3]=5+2=7`, i.e. `[2, 3, 5, 7]`. Scatter in reverse and the sorted array is `[0, 0, 1, 2, 2, 3, 3]`. The reverse scatter protects *stability*: by placing each value right-to-left within its block, equal keys retain their original relative order.

5. On the array labeled $[3_a, 1, 3_b, 2]$: **insertion**, **bubble**, and **counting** are stable; **selection** is not (its long-range swap can move 3_a past 3_b). A stable sort leaves the two 3s in the order 3_a then 3_b — the sorted result is $[1, 2, 3_a, 3_b]$.

References

[1] Ellis Horowitz and Sartaj Sahni. *Fundamentals of Data Structures*. Galgotia Booksource, 2nd edition, 2008.

[2] Narasimha Karumanchi. *Data Structures And Algorithms Made Easy*. CareerMonk Publications, 2017. This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1's C-specific pointer/malloc content — see this course's supporting-books index for the coverage check.

[3] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2nd edition, 1998.

[4] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, 4th edition, 2011.

[5] Niklaus Wirth. *Algorithms and Data Structures*. Author (freely distributed), 2004. Oberon version, August 2004; revision of the 1985 Prentice-Hall edition. Page numbers cited in CS301 material refer to this Oberon PDF, not to the 1985 print edition.

Concepts That Click